

共有AR体験の実装

Collaborative Session & MultipeerConnectivity

2025年1月

ydev勉強会

目次

目次

1. 共有AR体験とは
2. ARKitの共有機能の歴史
3. ARWorldMap による空間共有
4. Collaborative Session（協調セッション）
5. MultipeerConnectivity による通信
6. 実装例：共有ARアプリの構築
7. ユースケースと応用例

まとめ

参考リンク

共有AR体験の実装

Collaborative Session & MultipeerConnectivity

ydev勉強会

2025年1月

目次

- 1. 共有AR体験とは
- 2. ARKitの共有機能の歴史
- 3. ARWorldMap による空間共有
- 4. Collaborative Session（協調セッション）
- 5. MultipeerConnectivity による通信
- 6. 実装例：共有ARアプリの構築
- 7. ユースケースと応用例

1. 共有AR体験とは

概要

共有AR体験（Shared AR Experience）とは、複数のデバイスが同じAR空間を共有し、同じ仮想オブジェクトを見たり操作したりできる機能です。これはVision Proの「SharePlay」や空間共有機能のiOS版とも言えます。

主な特徴

特徴	説明
空間の同期	複数デバイスが同じ物理空間を認識
アンカーの共有	配置したオブジェクトの位置を共有
リアルタイム更新	一方の変更が他方にも即座に反映
ローカル通信	インターネット不要、近距離で動作

Vision Pro との比較

機能	Vision Pro (visionOS)	iOS ARKit
空間共有	SharePlay、空間ペルソナ	Collaborative Session
通信方式	FaceTime、GroupActivities	MultipeerConnectivity
同期精度	センチメートル単位	センチメートル単位
最大参加者	複数人	実質的に数人程度
空間永続性	クラウド連携	セッション中のみ

2. ARKitの共有機能の歴史

進化の過程

バージョン	年	機能
ARKit 2.0	2018	ARWorldMap による空間共有
ARKit 3.0	2019	Collaborative Session（自動協調）
ARKit 4.0	2020	ARGeoAnchor（位置情報アンカー）
ARKit 5.0+	2021～	精度向上、安定性改善

ARKit 2.0 : ARWorldMap

最初の共有機能。空間マップを手動でエクスポート・インポートする方式。

- ・ **利点:** 明示的な制御が可能
- ・ **欠点:** 手動での同期が必要、リアルタイム性に欠ける

ARKit 3.0 : Collaborative Session

自動的に空間情報を同期する高度な機能。

- ・ **利点:** 自動同期、リアルタイム、簡単な実装
- ・ **欠点:** より多くの通信帯域を使用

3. ARWorldMap による空間共有

ARWorldMap とは

ARWorldMapは、ARKitが認識した空間の「スナップショット」です。以下の情報を含みます：

- ・ 特徴点（Feature Points）の3D位置
- ・ 検出された平面
- ・ 配置されたアンカー
- ・ 空間の相対的な座標系

空間マップの取得

```
import ARKit

func captureWorldMap(session: ARSession, completion: @escaping (ARWorldMap?) -> Void) {
    session.getCurrentWorldMap { worldMap, error in
        if let error = error {
            print("WorldMap取得エラー: \(error.localizedDescription)")
            completion(nil)
            return
        }
        completion(worldMap)
    }
}
```

空間マップのシリアル化

```
func serializeWorldMap(_ worldMap: ARWorldMap) -> Data? {
    do {
        let data = try NSKeyedArchiver.archivedData(
            withRootObject: worldMap,
            requiringSecureCoding: true
        )
        return data
    } catch {
        print("シリアル化エラー: \(error)")
        return nil
    }
}

func deserializeWorldMap(from data: Data) -> ARWorldMap? {
    do {
        let worldMap = try NSKeyedUnarchiver.unarchivedObject(
            ofClass: ARWorldMap.self,
            from: data
        )
        return worldMap
    } catch {
        print("デシリアル化エラー: \(error)")
        return nil
    }
}
```

共有された空間の読み込み

```
func loadSharedWorldMap(_ worldMap: ARWorldMap, session: ARSession) {
    let configuration = ARWorldTrackingConfiguration()
    configuration.initialWorldMap = worldMap

    // 平面検出も有効化
    configuration.planeDetection = [.horizontal, .vertical]

    session.run(configuration, options: [.resetTracking, .removeExistingAnchors])
}
```

ARWorldMap の状態確認

```
func session(_ session: ARSession, didUpdate frame: ARFrame) {
    switch frame.worldMappingStatus {
    case .notAvailable:
        // マッピング不可
        print("空間マッピングが利用できません")
    case .limited:
        // 限定的なマッピング
        print("マッピング中...")
    case .extending:
        // マッピング拡張中
        print("空間を拡張中...")
    case .mapped:
        // マッピング完了 - 共有可能
        print("マッピング完了 - 共有準備OK")
    @unknown default:
        break
    }
}
```

4. Collaborative Session（協調セッション）

概要

ARKit 3.0で導入された Collaborative Session は、複数デバイス間でリアルタイムに空間情報を自動同期する機能です。ARWorldMapと異なり、手動でのマップ交換が不要です。

基本設定

```
import ARKit

func setupCollaborativeSession() -> ARWorldTrackingConfiguration {
    let configuration = ARWorldTrackingConfiguration()

    // 協調セッションを有効化
    configuration.isCollaborationEnabled = true

    // 平面検出
    configuration.planeDetection = [.horizontal, .vertical]

    // 環境テクスチャリング（オプション）
    configuration.environmentTexturing = .automatic

    return configuration
}
```

協調データの送信

```
extension YourARViewController: ARSessionDelegate {

    func session(_ session: ARSession, didOutputCollaborationData data: ARSession.CollaborationData) {
        // 協調データが生成されたら他デバイスに送信
        guard let encodedData = try? NSKeyedArchiver.archivedData(
            withRootObject: data,
            requiringSecureCoding: true
        ) else { return }

        // MultipeerConnectivity経由で送信
        sendToAllPeers(encodedData)
    }
}
```

協調データの受信と適用

```
func receivedCollaborationData(_ data: Data) {
    guard let collaborationData = try? NSKeyedUnarchiver.unarchivedObject(
        ofClass: ARSession.CollaborationData.self,
        from: data
    ) else {
        print("協調データのデコードに失敗")
        return
    }

    // ARSessionに協調データを適用
    arSession.update(with: collaborationData)
}
```

協調データの優先度

```
func session(_ session: ARSession, didOutputCollaborationData data: ARSession.CollaborationData) {
    switch data.priority {
    case .critical:
        // 重要なデータ（新しいアンカーなど） - 即座に送信
        sendImmediately(data)
    case .optional:
        // オプションのデータ（精度向上など） - バッチ送信可
        queueForBatchSend(data)
    @unknown default:
        sendImmediately(data)
    }
}
```


5. MultipeerConnectivity による通信

概要

MultipeerConnectivity は、Wi-Fi やBluetooth を使用して近くのデバイスと直接通信するAppleのフレームワークです。インターネット接続なしで動作します。

基本コンポーネント

コンポーネント	役割
MCPeerID	デバイスの識別子
MCSession	通信セッションの管理
MCNearbyServiceAdvertiser	自分のサービスを広告
MCNearbyServiceBrowser	近くのサービスを検索

MultipeerConnectivity マネージャーの実装

```
import MultipeerConnectivity

class MultipeerManager: NSObject, ObservableObject {

    // サービスタイプ (最大15文字、小文字とハイフンのみ)
    private let serviceType = "ar-collab"

    private let myPeerID: MCPeerID
    private var session: MCSession!
    private var advertiser: MCNearbyServiceAdvertiser!
    private var browser: MCNearbyServiceBrowser!

    @Published var connectedPeers: [MCPeerID] = []

    // データ受信時のコールバック
    var onDataReceived: ((Data, MCPeerID) -> Void)?

    override init() {
        // デバイス名でPeerIDを作成
        myPeerID = MCPeerID(displayName: UIDevice.current.name)
        super.init()

        setupSession()
    }

    private func setupSession() {
        // セッションの作成
        session = MCSession(
            peer: myPeerID,
            securityIdentity: nil,
            encryptionPreference: .required
        )
        session.delegate = self

        // アドバタイザーの作成 (自分を発見可能にする)
        advertiser = MCNearbyServiceAdvertiser(
            peer: myPeerID,
            discoveryInfo: nil,
            serviceType: serviceType
        )
        advertiser.delegate = self

        // ブラウザの作成 (他デバイスを探す)
        browser = MCNearbyServiceBrowser(
            peer: myPeerID,
            serviceType: serviceType
        )
        browser.delegate = self
    }

    func startHosting() {
        advertiser.startAdvertisingPeer()
        browser.startBrowsingForPeers()
    }
}
```

```
func stopHosting() {  
    advertiser.stopAdvertisingPeer()  
    browser.stopBrowsingForPeers()  
}  
  
func send(_ data: Data) {  
    guard !session.connectedPeers.isEmpty else { return }  
  
    do {  
        try session.send(data, toPeers: session.connectedPeers, with: .reliable)  
    } catch {  
        print("送信エラー: \(error)")  
    }  
}
```

MCSessionDelegate の実装

```
extension MultipeerManager: MCSessionDelegate {

    func session(_ session: MCSession, peer peerID: MCPeerID, didChange state: MCSessionState) {
        DispatchQueue.main.async {
            self.connectedPeers = session.connectedPeers

            switch state {
            case .notConnected:
                print("\(peerID.displayName) が切断されました")
            case .connecting:
                print("\(peerID.displayName) に接続中...")
            case .connected:
                print("\(peerID.displayName) が接続されました")
            @unknown default:
                break
            }
        }
    }

    func session(_ session: MCSession, didReceive data: Data, fromPeer peerID: MCPeerID) {
        // 受信データを処理
        DispatchQueue.main.async {
            self.onDataReceived?(data, peerID)
        }
    }

    // 以下は今回使用しないが、プロトコル準拠のため必要
    func session(_ session: MCSession, didReceive stream: InputStream, withName streamName: String, fromPeer peerID: MCPeerID) {}
    func session(_ session: MCSession, didStartReceivingResourceWithName resourceName: String, fromPeer peerID: MCPeerID, with progress: Progress) {}
    func session(_ session: MCSession, didFinishReceivingResourceWithName resourceName: String, fromPeer peerID: MCPeerID, at localURL: URL?, withError error: Error?) {}
}
```

MCMNearbyServiceAdvertiserDelegate の実装

```
extension MultipeerManager: MCMNearbyServiceAdvertiserDelegate {

    func advertiser(_ advertiser: MCMNearbyServiceAdvertiser, didReceiveInvitationFromPeer peerID: MCPeerID, withContext context: Data?, invitationHandler: @escaping (Bool, MCSession?) -> Void) {
        // 招待を自動承認（本番では確認UIを表示することを推奨）
        invitationHandler(true, session)
    }
}
```

MCNearbyServiceBrowserDelegate の実装

```
extension MultipeerManager: MCNearbyServiceBrowserDelegate {  
  
    func browser(_ browser: MCNearbyServiceBrowser, foundPeer peerID: MCPeerID, withDiscoveryInfo info: [String : String]?) {  
        // 発見したピアに招待を送信  
        browser.invitePeer(peerID, to: session, withContext: nil, timeout: 30)  
    }  
  
    func browser(_ browser: MCNearbyServiceBrowser, lostPeer peerID: MCPeerID) {  
        print("\(peerID.displayName) が見つからなくなりました")  
    }  
}
```

6. 実装例：共有ARアプリの構築

プロジェクト構成

```
SharedARApp/  
├─ App/  
│   └─ SharedARApp.swift  
├─ Views/  
│   └─ ContentView.swift  
├─ AR/  
│   ├── ARCoordinator.swift  
│   └─ SharedAnchorManager.swift  
├─ Networking/  
│   └─ MultipeerManager.swift  
└─ Info.plist
```

Info.plist の設定

MultipeerConnectivity を使用するには、Info.plist に以下を追加：

```
<key>NSLocalNetworkUsageDescription</key>  
<string>近くのデバイスとAR体験を共有するために使用します</string>  
  
<key>NSBonjourServices</key>  
<array>  
    <string>_ar-collab._tcp</string>  
    <string>_ar-collab._udp</string>  
</array>
```

ContentView.swift

```
import SwiftUI
import RealityKit
import ARKit

struct ContentView: View {
    @StateObject private var multipeerManager = MultipeerManager()
    @State private var arView: ARView?

    var body: some View {
        ZStack {
            ARViewContainer(
                multipeerManager: multipeerManager,
                arView: $arView
            )
                .edgesIgnoringSafeArea(.all)

            VStack {
                // 接続状態の表示
                HStack {
                    Circle()
                        .fill(multipeerManager.connectedPeers.isEmpty ? .red : .green)
                        .frame(width: 12, height: 12)

                    Text(multipeerManager.connectedPeers.isEmpty
                        ? "接続待機中..."
                        : "\((multipeerManager.connectedPeers.count)台接続中)")
                        .foregroundColor(.white)
                }
                .padding()
                .background(Color.black.opacity(0.6))
                .cornerRadius(8)

                Spacer()

                // オブジェクト配置ボタン
                Button(action: placeObject) {
                    Text("オブジェクトを配置")
                        .padding()
                        .background(Color.blue)
                        .foregroundColor(.white)
                        .cornerRadius(10)
                }
                .padding(.bottom, 50)
            }
            .padding()
        }
        .onAppear {
            multipeerManager.startHosting()
        }
    }

    private func placeObject() {
        guard let arView = arView else { return }

        // カメラの前方1mにオブジェクトを配置
    }
}
```



```
let anchor = AnchorEntity(world: [0, 0, -1])

let mesh = MeshResource.generateBox(size: 0.1)
let material = SimpleMaterial(color: .systemBlue, isMetallic: true)
let modelEntity = ModelEntity(mesh: mesh, materials: [material])

anchor.addChild(modelEntity)
arView.scene.addAnchor(anchor)
}
```

ARViewContainer.swift

```
import SwiftUI
import RealityKit
import ARKit

struct ARViewContainer: UIViewRepresentable {
    let multipeerManager: MultipeerManager
    @Binding var arView: ARView?

    func makeUIView(context: Context) -> ARView {
        let arView = ARView(frame: .zero)

        // Collaborative Session の設定
        let configuration = ARWorldTrackingConfiguration()
        configuration.isCollaborationEnabled = true
        configuration.planeDetection = [.horizontal]

        arView.session.delegate = context.coordinator
        arView.session.run(configuration)

        // 協調データ受信の設定
        multipeerManager.onDataReceived = { data, peer in
            context.coordinator.receiveData(data, from: peer)
        }

        DispatchQueue.main.async {
            self.arView = arView
        }

        return arView
    }

    func updateUIView(_ uiView: ARView, context: Context) {}

    func makeCoordinator() -> Coordinator {
        Coordinator(multipeerManager: multipeerManager)
    }

    class Coordinator: NSObject, ARSessionDelegate {
        let multipeerManager: MultipeerManager
        weak var arSession: ARSession?

        init(multipeerManager: MultipeerManager) {
            self.multipeerManager = multipeerManager
        }

        // 協調データの送信
        func session(_ session: ARSession, didOutputCollaborationData data: ARSession.CollaborationData) {
            self.arSession = session

            guard let encodedData = try? NSKeyedArchiver.archivedData(
                withRootObject: data,
                requiringSecureCoding: true
            ) else { return }
        }
    }
}
```

```
        multipeerManager.send(encodedData)
    }

    // 協調データの受信
    func receivedData(_ data: Data, from peer: MCPeerID) {
        guard let collaborationData = try? NSKeyedUnarchiver.unarchivedObject(
            ofClass: ARSession.CollaborationData.self,
            from: data
        ) else { return }

        arSession?.update(with: collaborationData)
    }
}
```

7. ユースケースと応用例

ゲーム

アプリ例	説明
ARボードゲーム	テーブルに仮想のゲーム盤を表示
対戦シューティング	同じ空間でAR的を撃ち合う
協力パズル	複数人で協力してパズルを解く

教育・トレーニング

アプリ例	説明
解剖学学習	複数の学生が同じ3Dモデルを観察
機械操作訓練	複数人でAR機械の操作を学習
歴史学習	歴史的建造物を共同で探索

ビジネス

アプリ例	説明
建築レビュー	現場で3Dモデルを共同確認
製品プレゼン	クライアントと3D製品を共有
空間デザイン	インテリア配置を共同検討

エンターテイメント

アプリ例	説明
AR美術館	複数人で仮想展示を鑑賞
共有フォトスポット	ARオブジェクトと一緒に記念撮影
ライブイベント	観客全員が同じAR演出を体験

まとめ

iOS の ARKit は、Vision Pro のような空間共有体験を実現するための強力な機能を提供しています。

主要ポイント

1. **ARWorldMap**: 空間マップの明示的な共有
2. **Collaborative Session**: 自動的なリアルタイム同期
3. **MultipeerConnectivity**: インターネット不要のP2P通信

次のステップ

- ・ 本プロジェクトに共有機能を実装
- ・ UI/UXの改善（招待フロー、接続状態表示）
- ・ パフォーマンスの最適化
- ・ エラーハンドリングの強化

参考リンク

- ・ [ARKit - Apple Developer](#)
- ・ [MultipeerConnectivity - Apple Developer](#)
- ・ [Creating a Collaborative Session - Apple Developer](#)
- ・ [SwiftUI and ARKit Integration](#)